

Step 04 - Fit a Regression Model

A First Practical Machine Learning Workflow

What this document is for

Step 03 split the tiny house-price dataset into training and test portions. Step 04 introduces the first model and uses only the training portion to teach it a simple relationship between house features and price.

By the end of this step, you should have:

- Created a blank linear-regression model object.
- Used **fit()** to train that model on **X_train** and **y_train** only.
- Inspected the learned intercept and feature weights without over-trusting them.

1. Where this step sits

This is the first document where a model actually learns from data. Up to now, the notebook has only created rows, chosen features, chosen the target, and split the examples.

Step	What changes
01 - Get the notebook ready	The environment and package checks exist.
02 - Create a tiny dataset	Rows, features, target, X, and y exist.
03 - Split before training	X_train, X_test, y_train, and y_test exist.
04 - Fit a regression model	A model learns from the training portion only.
05 - Measure wrongness	Predictions are compared with actual values using errors and loss.

The rule for this whole step

Train on X_train and y_train. Do not fit the model on X_test or y_test. The test rows are being saved for judging the model later.

2. What regression means here

Regression is the machine-learning task of predicting a number. In this tiny example, the number is **price_k**: the house price measured in thousands.

Task type	Predicts	House-price example
Regression	A number	Predict price_k, such as 335 or 520.

Classification	A category	Predict cheap / medium / expensive, or spam / not spam.
----------------	------------	---

Why start with linear regression?

It is readable. The model learns one weight for each feature, plus an intercept. That makes it a useful first model before less transparent models appear later.

3. The idea before the code

Simple schoolbook linear regression has one input and looks like $y = mx + c$. Our version has several input features, so the model learns several weights instead of one slope.

```
predicted_price = (
    intercept
    + bedrooms_weight * bedrooms
    + floor_area_weight * floor_area_m2
    + distance_weight * distance_station_km
    + age_weight * age_years
)
```

Interesting aside - why this resembles a perceptron

This has the same basic shape as a perceptron or a single artificial neuron. That is not a coincidence: the shared primitive is a **weighted sum of inputs plus a bias/intercept**.

Linear regression uses that weighted sum directly as a number. A perceptron takes a very similar weighted sum and then applies a threshold to make a class decision. Logistic regression pushes the weighted sum through a sigmoid to produce a probability. A neural-network neuron usually passes the weighted sum through an activation function.

So this is a useful landmark, not a new topic to chase yet. For now, keep the main idea: features are multiplied by learned weights, the intercept/bias is added, and the result becomes the model output or feeds the next decision step.

Model idea	Weighted sum is followed by...	Typical output
Linear regression	Nothing extra for this example	A continuous number, such as a price
Perceptron	A threshold / step decision	One of two classes
Logistic regression	A sigmoid curve	A probability for a class
Neural-network neuron	An activation function	A signal passed onward

Do not over-read the weights yet

The dataset has only 12 rows. The weights are useful for seeing that fitting created internal numbers, but they are not evidence about the real housing market.

4. Import one model class only when it is needed

The notebook now needs one new class from scikit-learn: **LinearRegression**. scikit-learn often calls model-like objects estimators. For now, you can read estimator as model object.

Why this cell exists

This is the first point where a model class is needed. The import does not create or train a model by itself; it only makes the class available.

Code cell

```
from sklearn.linear_model import LinearRegression
```

What to look for

The cell should run silently. You now have access to the **LinearRegression** class, but no model has learned anything yet.

5. Create a blank model object

Creating the object chooses the kind of model. It does not train it. At this point, the model is more like an empty tool than a learned result.

Why this cell exists

This creates one **LinearRegression** instance and gives it a meaningful notebook name: **house_model**.

Code cell

```
house_model = LinearRegression()
```

What to look for

The variable **house_model** now exists. It has not seen **X_train** or **y_train** yet, so it has not learned feature weights yet.

6. Fit the model on training data only

fit() is the training step. The model receives the training feature rows and their known prices, then estimates the internal numbers it needs for prediction.

Why this cell exists

This is where learning happens. The model may learn from **X_train** and **y_train**. It must not learn from **X_test** or **y_test**.

Code cell

```
house_model.fit(X_train, y_train)
```

What to look for

Jupyter will usually display something like **LinearRegression()**. That is not the prediction; it is just the fitted object being shown back to you.

Checkpoint

In your own words: **fit()** means use the training examples to learn internal model parameters. For linear regression, those parameters include feature weights and an intercept.

7. Inspect what fitting created

Linear regression is useful at this stage because we can peek inside it. The intercept is the baseline part of the prediction. The coefficients are the learned weights attached to the feature columns.

Why this cell exists

This uses pandas, which was imported in Step 02, to display one learned coefficient beside each feature name. The **feature_columns** list also came from Step 02.

Code cell

```
coefficients = pd.Series(
    data=house_model.coef_,
    index=feature_columns,
    name="learned_weight",
)

print("Intercept:", round(house_model.intercept_, 2))
coefficients.round(2)
```

What to look for

You should see one intercept value and four learned weights: one for **bedrooms**, one for **floor_area_m2**, one for **distance_station_km**, and one for **age_years**.

Avoid this common mistake

Do not treat these numbers as real-world truth. With only 8 training rows, the model is mostly a learning instrument. The point is to connect features, weights, and predictions.

8. Predict the held-back rows

predict() is different from **fit()**. **fit()** learns from known answers. **predict()** uses the already-fitted model to produce answers for feature rows.

Why this cell exists

Now we use **X_test**, the feature rows held back in Step 03. These rows were not used during fitting, so their predictions give us the material needed for a later evaluation.

Code cell

```
house_predictions = house_model.predict(X_test)

comparison = pd.DataFrame({
    "actual_price_k": y_test,
    "predicted_price_k": house_predictions.round(1),
})

comparison
```

What to look for

Each held-back row now has two values: the actual price from **y_test** and the model's predicted price. Do not worry yet about exactly how wrong the predictions are; Step 05 handles that deliberately.

Stop here before Step 05

You now have a trained regression model and predictions for held-back rows. The next document will turn actual-minus-predicted differences into residuals, errors, and loss.

Terms to Say Out Loud

Step 04 - Fit a Regression Model

Retrieval target

You do not need a dictionary definition. You should be able to say each term in plain English and connect it to the notebook work.

Term	Say it out loud as...
regression	A machine-learning task where the prediction is a number.
LinearRegression	The scikit-learn model class used for the first regression example.
estimator	scikit-learn's general word for a model-like object.
fit()	Train the model using training features and known training targets.
predict()	Use the fitted model to produce answers for feature rows.
intercept	The baseline part of a linear regression prediction.
coefficient / weight	A learned number attached to a feature.
weighted sum	Features multiplied by weights, then combined with an intercept or bias.
bias	Another common word for the intercept-like offset in a model.